여수 autoencoder

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

df1 = pd.read_excel("2023.xlsx")
df2 = pd.read_excel("2024.xlsx")
df3 = pd.read_excel("2025.xlsx")

df = pd.concat([df1, df2, df3], ignore_index=True)

df = df.rename(columns={
    '누적강수량(mm)': 'rainfall',
    '일시': 'datetime'
})

df['datetime'] = pd.to_datetime(df['datetime'])
df = df.sort_values('datetime')
```

```
df = df.set_index('datetime')
df = df.resample('30min').mean()
df = df.reset_index()

np.random.seed(42)

num_events = 8
rain_indices = np.random.choice(len(df), size=num_events, replace=False)

for idx in rain_indices:

    duration = np.random.choice([6, 12, 24, 36])

    heavy_rain = np.random.choice([30, 50, 80, 100])

    for i in range(idx, min(idx + duration, len(df))):
        df.loc[i, 'rainfall'] = heavy_rain

features = ['rainfall']

for col in ['평균기온(℃)', '습도(%)', '풍속(m/s)', '해면기압(hPa)', '현지기압(hPa)']:
    if col in df.columns:
        features.append(col)

print("사용  변수:", features)

df = df.fillna(df.mean(numeric_only=True))
```

```
data = df[features].values

scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)

X = []
n_past = 5

for i in range(n_past, len(scaled_data)):
    X.append(scaled_data[i - n_past:i].flatten())

X = np.array(X)

print("X shape:", X.shape)

input_dim = X.shape[1]

input_layer = Input(shape=(input_dim,))

encoded = Dense(16, activation='relu')(input_layer)
encoded = Dense(8, activation='relu')(encoded)

decoded = Dense(16, activation='relu')(encoded)
decoded = Dense(input_dim, activation='linear')(decoded)

autoencoder = Model(inputs=input_layer, outputs=decoded)
```

```
autoencoder.compile(optimizer='adam', loss='mse')

history = autoencoder.fit(
    X, X,
    epochs=30,
    batch_size=32,
    verbose=1
)

reconstructed = autoencoder.predict(X)

errors = np.mean(np.power(X - reconstructed, 2), axis=1)

actual = X[:, 0]
predicted = reconstructed[:, 0]

mse = mean_squared_error(actual, predicted)
print("AutoEncoder MSE:", round(mse, 6))

plt.figure(figsize=(10,5))

plt.plot(actual[-300:], label='Actual')
plt.plot(predicted[-300:], label='Reconstructed')

plt.title("AutoEncoder Reconstruction (Long Heavy Rain)")
plt.xlabel("Time")
```

```
plt.ylabel("Scaled")


plt.legend()
plt.grid()
plt.show()


plt.figure(figsize=(6,6))


plt.scatter(actual, predicted, alpha=0.3)


min_val = min(actual.min(), predicted.min())
max_val = max(actual.max(), predicted.max())


plt.plot([min_val, max_val], [min_val, max_val], color='red')


plt.title("AutoEncoder Performance")
plt.xlabel("Actual")
plt.ylabel("Reconstructed")
plt.grid()
plt.show()


threshold = np.percentile(errors, 95)


plt.figure(figsize=(10,5))


plt.plot(errors, label='Reconstruction Error')
```

```
plt.axhline(threshold, color='red', linestyle='--', label='Threshold')


anomalies = np.where(errors > threshold)[0]
plt.scatter(anomalies, errors[anomalies], color='red', label='Anomaly')


plt.title("Anomaly Detection (Long Heavy Rain)")
plt.xlabel("Time")
plt.ylabel("Error")


plt.legend()
plt.grid()
plt.show()
```

통영 autoencoder

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf


from sklearn.preprocessing import StandardScaler

from sklearn.metrics import mean_squared_error


from tensorflow.keras.models import Model

from tensorflow.keras.layers import Input, Dense


df1 = pd.read_excel("to 2023.xlsx")

df2 = pd.read_excel("to 2024.xlsx")

df3 = pd.read_excel("to 2025.xlsx")


df = pd.concat([df1, df2, df3], ignore_index=True)


df = df.rename(columns={

    '누적강수량(mm)': 'rainfall',

    '일시': 'datetime'

})


df['datetime'] = pd.to_datetime(df['datetime'])
```

```
df = df.sort_values('datetime')


df = df.set_index('datetime')
df = df.resample('30min').mean()
df = df.reset_index()


np.random.seed(42)


num_events = 8
rain_indices = np.random.choice(len(df), size=num_events, replace=False)


for idx in rain_indices:
    duration = np.random.choice([6, 12, 24, 36])
    heavy_rain = np.random.choice([30, 50, 80, 100])


    for i in range(idx, min(idx + duration, len(df))):
        df.loc[i, 'rainfall'] = heavy_rain



features = ['rainfall']


for col in ['평균기온(°C)', '습도(%)', '풍속(m/s)', '해면기압(hPa)', '현지기압(hPa)']:
    if col in df.columns:
        features.append(col)


print("사용  변수:", features)
```

```
df = df.fillna(df.mean(numeric_only=True))



# ============================
# 7. 스케일링
# ============================
data = df[features].values

scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)

X = []
n_past = 20

for i in range(n_past, len(scaled_data)):
    window = scaled_data[i - n_past:i].copy()

    diff = np.diff(window[:, 0], prepend=window[0, 0]).reshape(-1,1)

    window = np.hstack([window, diff])

    X.append(window.flatten())

X = np.array(X)

input_dim = X.shape[1]
```

```
input_layer = Input(shape=(input_dim,))

encoded = Dense(64, activation='relu')(input_layer)
encoded = Dense(32, activation='relu')(encoded)

decoded = Dense(64, activation='relu')(encoded)
decoded = Dense(input_dim, activation='linear')(decoded)

autoencoder = Model(inputs=input_layer, outputs=decoded)

autoencoder.compile(optimizer='adam', loss='mse')

autoencoder.fit(
    X, X,
    epochs=30,
    batch_size=32,
    verbose=1
)


reconstructed = autoencoder.predict(X)

actual = X[:, -features.__len__()-1]   # rainfall 위치
predicted = reconstructed[:, -features.__len__()-1]

def inverse_rainfall(series, scaler):
```

```
    return series * scaler.scale_[0] + scaler.mean_[0]

actual_inv = inverse_rainfall(actual, scaler)
pred_inv = inverse_rainfall(predicted, scaler)

mse = mean_squared_error(actual_inv, pred_inv)
print("MSE:", int(mse))

plt.figure(figsize=(10,5))

plt.plot(actual_inv[-300:], label='Actual')
plt.plot(pred_inv[-300:], label='Reconstructed')

plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(6,6))

plt.scatter(actual_inv, pred_inv, alpha=0.3)

m = min(actual_inv.min(), pred_inv.min())
M = max(actual_inv.max(), pred_inv.max())

plt.plot([m, M], [m, M], color='red')
plt.grid()
plt.show()
```

```
errors = np.mean((X - reconstructed)**2, axis=1)


threshold = np.percentile(errors, 99)


plt.figure(figsize=(10,5))

plt.plot(errors)

plt.axhline(threshold, color='red')


anomalies = np.where(errors > threshold)[0]

plt.scatter(anomalies, errors[anomalies], color='red')


plt.grid()

plt.show()
```

여수 SVR

```
# 1. 라이브러리
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
from sklearn.svm import SVR

df1 = pd.read_excel("2023.xlsx")
df2 = pd.read_excel("2024.xlsx")
df3 = pd.read_excel("2025.xlsx")

df = pd.concat([df1, df2, df3], ignore_index=True)

df = df.rename(columns={
    '누적강수량(mm)': 'rainfall',
    '일시': 'datetime'
})

df['datetime'] = pd.to_datetime(df['datetime'])
df = df.sort_values('datetime')

df = df.set_index('datetime')
```

```
df = df.resample('30T').mean()
df = df.reset_index()

features = ['rainfall']

for col in ['평균기온(°C)', '습도(%)', '풍속(m/s)', '해면기압(hPa)', '현지기압(hPa)']:
    if col in df.columns:
        features.append(col)

print("사용 변수:", features)

df = df.fillna(df.mean(numeric_only=True))

data = df[features].values

scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)

X = []
y = []
n_past = 5    # 30분 기준이면 2.5시간

for i in range(n_past, len(scaled_data)):
    X.append(scaled_data[i - n_past:i].flatten())
    y.append(scaled_data[i, 0])

X = np.array(X)
```

```
y = np.array(y)

model = SVR(
    kernel='rbf',
    C=100,
    gamma=0.1,
    epsilon=0.1
)

model.fit(X, y)

predictions = model.predict(X)

mse = mean_squared_error(y, predictions)
print("SVR MSE:", round(mse, 6))

plt.figure(figsize=(10,5))

plt.plot(y[-300:], label='Actual')
plt.plot(predictions[-300:], label='Predicted')

plt.title("SVR Prediction (30-minute average)")
plt.xlabel("Time")
plt.ylabel("Scaled")

plt.legend()
plt.grid()
```

```
plt.show()



plt.figure(figsize=(6,6))


plt.scatter(y, predictions, alpha=0.3)


min_val = min(y.min(), predictions.min())

max_val = max(y.max(), predictions.max())


plt.plot([min_val, max_val], [min_val, max_val], color='red')


plt.title("SVR Performance (30-min Avg)")

plt.xlabel("Actual")

plt.ylabel("Predicted")

plt.grid()

plt.show()
```

통영 SVR

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
from sklearn.svm import SVR



df1 = pd.read_excel("to 2023.xlsx")
df2 = pd.read_excel("to 2024.xlsx")
df3 = pd.read_excel("to 2025.xlsx")

df = pd.concat([df1, df2, df3], ignore_index=True)

df = df.rename(columns={
    '누적강수량(mm)': 'rainfall',
    '일시': 'datetime'
})



df['datetime'] = pd.to_datetime(df['datetime'])
df = df.sort_values('datetime')
```

```
df = df.set_index('datetime')
df = df.resample('30T').mean()
df = df.reset_index()


features = ['rainfall']

for col in ['평균기온(°C)', '습도(%)', '풍속(m/s)', '해면기압(hPa)', '현지기압(hPa)']:
    if col in df.columns:
        features.append(col)

print("사용 변수:", features)


df = df.fillna(df.mean(numeric_only=True))


data = df[features].values

scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)


X = []
y = []
n_past = 5    # 30분 기준이면 2.5시간
```

```
for i in range(n_past, len(scaled_data)):
    X.append(scaled_data[i - n_past:i].flatten())
    y.append(scaled_data[i, 0])

X = np.array(X)
y = np.array(y)



model = SVR(
    kernel='rbf',
    C=100,
    gamma=0.1,
    epsilon=0.1
)

model.fit(X, y)



predictions = model.predict(X)



mse = mean_squared_error(y, predictions)
print("SVR MSE:", round(mse, 6))



plt.figure(figsize=(10,5))
```

```
plt.plot(y[-300:], label='Actual')
plt.plot(predictions[-300:], label='Predicted')

plt.title("SVR Prediction (30-minute average)")
plt.xlabel("Time")
plt.ylabel("Scaled")

plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(6,6))

plt.scatter(y, predictions, alpha=0.3)

min_val = min(y.min(), predictions.min())
max_val = max(y.max(), predictions.max())

plt.plot([min_val, max_val], [min_val, max_val], color='red')

plt.title("SVR Performance (30-min Avg)")
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.grid()
plt.show()
```